

# Multilingual AppWorld Vanilla Evaluation Pipeline

Technical Specification & Comprehensive Architecture Documentation

Danish Javed

[github.com/DanishTriesCoding/multilingual-appworld-vanilla-eval-pipeline](https://github.com/DanishTriesCoding/multilingual-appworld-vanilla-eval-pipeline)

August 15, 2026

## Abstract

This document provides the complete technical specification for the **Multilingual AppWorld Vanilla Evaluation Pipeline**, a modular, robust, direct-inference (zero-shot) evaluation harness for the **AppWorld** benchmark (StonyBrookNLP/appworld). The pipeline evaluates local open-weights Large Language Models (e.g., Qwen2.5, Llama-3, Mistral) operating as autonomous interactive coding agents in AppWorld's digital environment. It interfaces with any OpenAI-compatible inference server (vLLM, SGLang, Ollama, TGI), natively supports cross-lingual instruction evaluations via custom JSON task mappings, guarantees process isolation across parallel rollouts, and provides sandbox-safe persistence against environment monkey-patches.

## Contents

---

|          |                                               |          |
|----------|-----------------------------------------------|----------|
| <b>1</b> | <b>Introduction and Pipeline Overview</b>     | <b>1</b> |
| 1.1      | Key Design Principles . . . . .               | 2        |
| <b>2</b> | <b>Repository Architecture &amp; File Map</b> | <b>2</b> |
| <b>3</b> | <b>Formal Evaluation Metrics</b>              | <b>2</b> |
| 3.1      | Task Goal Completion (TGC) . . . . .          | 2        |
| 3.2      | Scenario Goal Completion (SGC) . . . . .      | 3        |
| 3.3      | Bootstrap Confidence Intervals . . . . .      | 3        |
| <b>4</b> | <b>Setup &amp; Installation</b>               | <b>3</b> |
| 4.1      | Virtual Environment Setup . . . . .           | 3        |
| 4.2      | AppWorld Installation and Data Sync . . . . . | 3        |
| <b>5</b> | <b>Subsampling vs. Full Split Execution</b>   | <b>4</b> |
| <b>6</b> | <b>Multilingual Evaluation Protocol</b>       | <b>4</b> |
| <b>7</b> | <b>Configuration Reference</b>                | <b>4</b> |
| <b>8</b> | <b>CLI Usage Reference</b>                    | <b>5</b> |
| <b>9</b> | <b>Troubleshooting &amp; Diagnostic Guide</b> | <b>5</b> |

## 1 Introduction and Pipeline Overview

---

The **AppWorld** benchmark tests autonomous agents across complex, multi-day digital tasks interacting with 9 simulated applications and  $\sim 457$  APIs. Evaluating LLMs in this environment requires iterative

interaction: the agent observes textual state feedback, generates executable Python code blocks, and completes tasks via designated supervisor API endpoints.

### 1.1 Key Design Principles

- **Model & Server Agnostic:** Any server providing a standard POST /v1/chat/completions endpoint is supported.
- **Native Multilingual Evaluation:** Direct mapping of translated instructions ( $T_{\text{target}}$ ) to canonical AppWorld task IDs without mutating base data files.
- **Process-Level Sandbox Isolation:** AppWorld utilizes sandbox hooks and stateful globals. The pipeline executes rollouts via multiprocessing with spawn context and process recycling (maxtasksperchild=1).
- **Sandbox-Resilient Storage:** Low-level POSIX file descriptors (os.open, os.write) bypass AppWorld’s process-wide open() read-only locks.
- **Statistically Sound Metrics:** Computes Task Goal Completion (avg@ $k$ , best@ $k$ , unbiased pass@ $k$ ), Scenario Goal Completion (SGC), and 95% bootstrap confidence intervals.

## 2 Repository Architecture & File Map

The repository separates environment interaction, agent policy, API communication, and metric reporting into isolated, modular components. Below is the clean directory tree structure:

```
MULTILINGUAL-APPWORLD-VANILLA-EVAL-PIPELINE/
|-- .gitignore           # Git ignore rules (caches, local trajectories)
|-- diag.py              # Rapid single-task rollout diagnostic utility
|-- export_instructions.py # Utility to dump split task instructions to JSON
|-- instructions_en.json  # Base extracted English task instructions map
|-- pyproject.toml        # Build metadata & console CLI entrypoints
|-- README.md            # Repository overview and setup guide
|-- requirements.txt      # Package dependencies (requests, PyYAML, appworld)
|
|-- configs/
|   |-- smoke_dev.yaml    # Lightweight 4-task dev smoke test config
|   |-- vanilla_qwen7b.yaml # Benchmark evaluation config (Qwen2.5-7B)
|
|-- prompts/
|   |-- appworld_official.txt # Canonical AppWorld prompt template
|
|-- scripts/
|   |-- run_full_eval.sh   # Bash automation for full evaluation sweeps
|   |-- selftest.py        # Offline, GPU-less pipeline validation suite
|
|-- src/
|   |-- appworld_vanilla/
|       |-- __init__.py    # Package initialization
|       |-- agent.py       # Message history management & step proposal policy
|       |-- cli.py         # CLI routers (check, sample, run, report, eval)
|       |-- config.py      # Typed dataclasses & YAML configuration parser
|       |-- env.py         # Isolated interface encapsulating appworld package
|       |-- llm_client.py  # OpenAI-compatible HTTP client with retries
|       |-- metrics.py     # avg@k, best@k, pass@k, SGC, & bootstrap CIs
|       |-- parsers.py     # Code block extraction & output truncation
|       |-- prompts.py     # Prompt constructors & multi-turn template parser
|       |-- report.py      # Terminal summary renderer & safe CSV exporter
|       |-- runner.py      # Single rollout execution & parallel sweep runner
|       |-- sampler.py     # Scenario-stratified & randomized task selection
|       |-- storage.py     # Thread-safe POSIX JSONL result store & trajectory dumps
```

## 3 Formal Evaluation Metrics

AppWorld groups tasks into scenarios where each scenario  $S_j$  contains 3 distinct, interdependent tasks  $\{t_{j,1}, t_{j,2}, t_{j,3}\}$ .

### 3.1 Task Goal Completion (TGC)

Let  $k$  be the number of rollouts per task, and  $y_{i,s} \in \{0, 1\}$  denote the binary evaluation outcome of task  $i$  on rollout seed  $s \in \{1, \dots, k\}$ .

- **Average Success Rate (avg@k):**

$$\text{avg@k} = \frac{1}{N \cdot k} \sum_{i=1}^N \sum_{s=1}^k y_{i,s} \quad (1)$$

- **Best-of- $k$  Success Rate (best@k):**

$$\text{best@k} = \frac{1}{N} \sum_{i=1}^N \max_{s \in \{1, \dots, k\}} y_{i,s} \quad (2)$$

- **Unbiased pass@k (Codex Estimator):** When  $n \geq k$  rollouts are generated and  $c_i = \sum_{s=1}^n y_{i,s}$  rollouts succeed for task  $i$ :

$$\text{pass@k} = \frac{1}{N} \sum_{i=1}^N \left[ 1 - \frac{\binom{n-c_i}{k}}{\binom{n}{k}} \right] = \frac{1}{N} \sum_{i=1}^N \left[ 1 - \prod_{j=0}^{k-1} \frac{n-c_i-j}{n-j} \right] \quad (3)$$

### 3.2 Scenario Goal Completion (SGC)

A scenario is solved under seed  $s$  if and only if all 3 constituent tasks succeed:

$$Y_{j,s}^{\text{scenario}} = \prod_{m=1}^3 y_{j,m,s} \quad (4)$$

- **Scenario avg@k:**  $\frac{1}{|S| \cdot k} \sum_{j=1}^{|S|} \sum_{s=1}^k Y_{j,s}^{\text{scenario}}$
- **Scenario best@k:**  $\frac{1}{|S|} \sum_{j=1}^{|S|} \left( \prod_{m=1}^3 \max_s y_{j,m,s} \right)$

### 3.3 Bootstrap Confidence Intervals

Non-parametric 95% bootstrap confidence intervals are computed over  $B = 2,000$  resamples of task clusters with replacement to ensure statistically sound comparisons between baseline models and multilingual setups.

## 4 Setup & Installation

### 4.1 Virtual Environment Setup

```
# Clone and enter directory
git clone https://github.com/DanishTriesCoding/multilingual-appworld-vanilla-eval-
  ↳ pipeline.git
cd multilingual-appworld-vanilla-eval-pipeline

# Initialize virtual environment (Python 3.10+)
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate

# Install editable package
pip install -e .
```

### 4.2 AppWorld Installation and Data Sync

```
pip install appworld>=0.1.3
appworld install
appworld download data
```



```

num_rollouts: 8                # k parameter
seed_base: 1000
max_workers: 4                 # Multi-process parallelism
output_dir: "results"
resume: true
task_ids_file: null            # Custom subset file path (null = full split)
instruction_map_file: null      # Multilingual mapping JSON

```

## 8 CLI Usage Reference

```

# 1. Health check LLM endpoint and AppWorld installation
appworld-vanilla check -c configs/smoke_dev.yaml

# 2. Run an offline self-test (GPU-less & data-less)
python scripts/selftest.py

# 3. Sample 36 scenario-stratified tasks
appworld-vanilla sample -c configs/vanilla_qwen7b.yaml \
  --set sample.size=36 \
  --out task_ids_36.txt

# 4. Execute evaluation sweep
appworld-vanilla run -c configs/vanilla_qwen7b.yaml \
  --set run.task_ids_file=task_ids_36.txt

# 5. Display evaluation report and export CSV
appworld-vanilla report -c configs/vanilla_qwen7b.yaml

# 6. Run official AppWorld evaluator over generated trajectories
appworld-vanilla official-eval -c configs/vanilla_qwen7b.yaml

```

## 9 Troubleshooting & Diagnostic Guide

- **All rollouts end with unparseable\_output:** The model is failing to output code fenced in ‘‘python blocks. Increase `agent.max_empty_code_retries` or refine the system prompt.
- **High occurrence of `max_steps_exhausted`:** Small zero-shot models often require  $\geq 25$  steps to explore API specifications and obtain authentication tokens. If `agent.max_steps` is set too low (e.g., 12 steps), the model runs out of turn budget before completing goals.
- **Evaluating entire split instead of a subset:** Ensure `--set run.task_ids_file=<filename>` is passed to `run`, or specify `task_ids_file` in the configuration YAML.
- **`avg@k == best@k` identically:** Verify that `llm.temperature` is greater than 0 and that seeds vary across rollouts.

## References

1. Chowdhury, H., et al. (2024). *AppWorld: A Controllable World of Apps and Execution Environment for Agent Evaluation*. In Proceedings of the Association for Computational Linguistics (ACL 2024).
2. Chen, M., et al. (2021). *Evaluating Large Language Models Trained on Code*. arXiv:2107.03374 (Codex pass@k formulation).